

AI-Based Face Recognition Attendance System

Comprehensive Project Guide, AI Architecture & Viva Presentation Reference

Executive Summary

This project is an automated, real-time face recognition attendance system designed to eliminate manual roll calls. Built with Python, OpenCV, SQLite, and Streamlit, it captures live video feeds via webcam, detects human faces, recognizes enrolled students using machine learning, logs attendance records into a relational database (ensuring one entry per student daily), and displays real-time reports on an interactive web dashboard.

1. What Kind of AI Does This Project Use?

This project is powered by Classical Artificial Intelligence, Computer Vision (CV), and Supervised Machine Learning (ML). It relies on two primary algorithms working sequentially in a real-time vision pipeline:

A. Face Detection: OpenCV Haar Cascade Classifier

Haar Cascade is a machine learning object detection algorithm (Viola & Jones, 2001) trained on thousands of positive face samples and negative background samples.

- Haar Features: Evaluates rectangular pixel intensity differences (e.g., eye regions are darker than cheekbones).
- Integral Images: Enables feature calculation in constant $O(1)$ time.
- AdaBoost Selection: Filters out weak features to retain only critical facial cues.
- Cascade Architecture: Quick rejection of non-face regions to maintain high FPS performance.

B. Face Recognition: Local Binary Patterns Histogram (LBPH)

LBPH is a supervised machine learning pattern recognition algorithm designed specifically for texture and facial structure classification.

1. 3x3 Pixel Neighborhood Thresholding: Compares each pixel with its 8 neighbors. Assigns 1 if neighbor $\geq$ center, else 0, creating an 8-bit binary number.
2. Decimal Conversion: Converts binary pattern into a decimal value (0-255).
3. Histogram Representation: Divides the face crop into a grid (e.g., 8x8 cells) and builds local spatial histograms.
4. Distance Metric (Matching): Calculates Chi-Square / Euclidean distance between live face histograms and trained model histograms in `trainer.yml`. Lower distance = higher match confidence.

Why Classical Machine Learning (Haar + LBPH) Over Deep Learning?

- Ultra Lightweight & Fast: Runs at 30+ FPS on standard laptop CPUs without GPU hardware.
- Windows Friendly: No dependency on heavy C++ build tools like dlib / CMake.
- Lighting Resilient: Local Binary Patterns and Histogram Equalization normalize uneven lighting.
- Tradeoff: Deep learning models (e.g., FaceNet) offer higher accuracy for extreme side profiles but require significantly higher computational resources.

2. Step-by-Step Execution Guide

Step 1: Install Dependencies

Open terminal in project folder and run:

```
pip install -r requirements.txt
```

Step 2: Register a New Student

Capture student details and webcam face samples:

```
python register_student.py
```

Input Roll Number and Name when prompted. Look at the webcam while it captures ~150 cropped face images into dataset/<label_id>_<roll_no>/.

Step 3: Train the Machine Learning Model

Train the LBPH face recognizer on captured samples:

```
python train_model.py
```

Generates trainer/trainer.yml (trained model weights) and trainer/labels.json (label mapping).

Step 4: Run Live Attendance Marking

Launch real-time webcam recognition and SQLite attendance logging:

```
python mark_attendance.py
```

- Recognized students appear with GREEN boxes and names.
- Unregistered faces appear with RED boxes as 'Unknown'.
- Press 'q' to exit webcam feed.

Step 5: Launch Web Dashboard

View interactive web UI and download CSV reports:

```
streamlit run app.py
```

3. Project Components & File Structure

- config.py - Central configuration storing directory paths, face target dimensions (200x200), Haar Cascade XML path, sample limit (150), and LBPH distance threshold (75).
- database.py - SQLite manager handling students and attendance tables. Implements UNIQUE(roll_no, date) constraint to prevent duplicate attendance logs.
- face_utils.py - Shared Computer Vision functions: face detection, cropping, resizing, cv2.equalizeHist() for lighting normalization, and model loader.
- register_student.py - CLI tool to register student profiles and record dataset face images.
- train_model.py - Compiles trained LBPH model weights from images in dataset/.
- mark_attendance.py - Real-time camera loop executing detection, prediction, bounding box overlays, and database logging.
- app.py - Streamlit UI dashboard featuring attendance summary, search by date, student directory, and CSV export.

4. Viva & Technical Interview Q&A

Q1: How does the system prevent duplicate attendance entries for the same student on the same day?

Answer: The SQLite attendance table defines a composite UNIQUE constraint on (roll_no, date). Any secondary detection on the same date triggers an 'INSERT OR IGNORE' query, ensuring exactly one record per student per day.

Q2: What is the role of cv2.equalizeHist() in preprocessing?

Answer: Histogram equalization spreads out the most frequent intensity values in grayscale images. It normalizes lighting variations, preventing shadows or bright sunlight from degrading recognition accuracy.

Q3: What does the confidence metric 'distance' mean in LBPH?

Answer: LBPH computes histogram dissimilarity distance. A value of 0 indicates an exact match. LOWER distance means HIGHER confidence. If distance exceeds threshold 75, it classifies the person as 'Unknown'.

Q4: How could this system be upgraded for anti-spoofing / liveness detection?

Answer: By adding eye-blink detection via facial landmarks (dlib/MediaPipe), head rotation prompts, or thermal/3D depth camera inputs to prevent photo or video spoofing.